Exercise 15

Meet Structs

For this exercise, you get to work with structs. You'll be defining a new struct type, dynamically allocating an array of that type of struct, filling the array with input from a file and then using it to answer a sequence of queries from the user. On the course homepage, you'll find a source file named gps.c, along with some input files and expected outputs for this exercise. You can also copy these files to your current directory with the following shell command, if you're logged in on remote.eos.ncsu.edu:

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise15/* .
```

Program Input

The gps.c program expects a single command-line argument, the name of an input file containing a list of cities, each with a global location. The input file starts with an integer, ***n***, telling you how many cities follow. Each city is given as a name, followed by two real numbers giving the latitude and longitude of the city. We're proving two sample input files, city-1.txt just contains two cities:

```
2
Raleigh 35.817 -78.65
Tokyo 35.683 139.7
```

The city-2.txt file is a little bit longer, with records for 10 cities:

```
10
Chicago 41.883 -87.633
New-York 40.667 -73.933
Houston 29.767 -95.383
Los-Angeles 34.05 -118.25
Philadelphia 39.95 -75.167
Raleigh 35.817 -78.65
Phoenix 33.45 -112.067
Seattle 47.617 -122.333
Miami 25.783 -80.217
Denver 39.733 -104.983
```

Sample Execution

When it starts up, your program will read the list of cities into a dynamically allocated array of structs (some of this code is already written for you). Then, it will repeatedly read queries from the user, with each query giving a floating point latitude and longitude and an integer count, ***c,*** for the number of results to report. For each query, the program will report the ***c*** cities that are closest to query location. The results should be sorted by distance to the query location (closest first) and, if ***c*** is larger than ***n***, then only ***n*** cities will be reported.

The report for each city should be in the format shown below. Each city is reported on a line, with the name of the city, then a space, then the distance to the query location in miles, rounded to the nearest hundredth of a mile. Running your program as follows should give you the same output as expected-1.txt (typing quit at the end terminates the program, since it stops when it gets input that doesn't parse as query):

```
$ ./gps city-1.txt
40.667 -73.933 1
Raleigh (421.56 miles)
37.567 126.983 1
Tokyo (716.54 miles)
```

```
quit
```

Or, you can try running your program on the larger city list;

```
$ ./gps city-2.txt
33.75 -84.383 1
Raleigh (355.24 miles)
36.183 -115.133 2
Los-Angeles (229.67 miles)
Phoenix (256.70 miles)
-11.95 -77.033 99
Miami (2616.04 miles)
Houston (3130.88 miles)
Raleigh (3302.27 miles)
Philadelphia (3588.17 miles)
New-York (3641.12 miles)
Chicago (3780.35 miles)
Phoenix (3893.01 miles)
Denver (3996.75 miles)
Los-Angeles (4174.10 miles)
Seattle (4967.07 miles)
quit
```

The examples above show how to execute the program interactively, with the output going to the terminal and the query locations typed in by the user. We've provided files that let you run the program with standard input from a file. This will make it easier to check your output against the provided expected output files. You can use commands like the following to run your program against the first test case:

```
$ ./gps city-1.txt < input-1.txt > output.txt
$ echo $?
0
$ diff output.txt expected-1.txt
```

Similar commands will let you try your program against the larger test input:

```
$ ./gps city-2.txt < input-2.txt > output.txt
$ echo $?
0
$ diff output.txt expected-2.txt
```

Completing the Implementation

In the **gps.c** file, I've already written some code to get you started. I've marked the places where you'll need to fill in missing code with comments containing "...". Specifically, you'll need to add code for the following:

- Define a struct named City, for holding information about a city. Your struct will need to be able to store a name for the city (a string up to 20 characters in length). It will also need two double fields for storing the city's location, as latitude and longitude.

- Fill in the body of the readCity() function. It should read the name and location of a city from a file and store it in a city struct (both of these are parameters to the function). The city name won't contain any spaces, and you won't need to worry about detecting invalid input (e.g., a city name that's too long or a location that isn't given as a pair of double values). We won't run your program with invalid input files.

- In main(), add code to dynamically allocate an array of City structs, with enough elements to hold all the cities in the input file. I've already written code to read the first integer, n, from the input file. You'll need to declare a pointer-to-city variable to keep up with the location of your dynamically allocated array.

- Right after you make your array of City structs, add code to loop through the array and call your readCity() function to read in each of them. You defined readCity() to take a pointer to a City as its second parameter. Each time you call it, you need to give it the address of one of the elements of your dynamically allocated array. You could do this with pointer arithmetic, if you want.

- Near the end of main, there's a loop to read query locations from the user. Inside this loop, add code to sort your list of cities using qsort(). You'll be sorting them based on their distance to the query location. Use the compareDist() function as the comparison function for your qsort().

- Up above the implementation for main(), fill in the body of the comapreDist() function so that it compares two cities based on their distance to the query point. To do this, you'll need to convert the generic pointers passed in from qsort() to pointers to City instances, so you can check the coordinates of the two cities you're supposed to compare. You can use the globalDistance() function to compute the approximate distance to the query location, then return -1, 0 or 1 based on which city is closer.

  Notice that the query coordinates are stored in global variables, **qLat** and **qLon**. This lets the compareDistance() access these values each time it's called. We can't pass in these values as parameters, since qsort() is expecting a comparison function that just takes two void pointers.

- Add code inside the query-processing loop to print a report of (up to) ***c*** cities. After the city list is sorted, you should be able to just report cities starting from the first ones on the list.

- Finally, don't forget to free your dynamically allocated array before you exit.

Compiling Your Solution

This program uses a few functions from the math library. When you compile, you'll need to add the -lm option to tell gcc to link with the math library, and you'll need the -D_GNU_SOURCE compiler flag to tell gcc to let you use the M_PI constant for pi. This constant is commonly supplied by the math.h header, but isn't officially part of the C99 standard.

```
$ gcc -D_GNU_SOURCE -Wall -std=c99 gps.c -o gps -lm
```

Submitting your Work

Once your program is working, submit it to the exercise_15 assignment in Moodle.